

NATIONAL INSTITUTE OF ELECTRONICS & INFORMATION TECHNOLOGY (NIELIT), ROPAR
Department of Computer Science Engineering (AI & ML)

Student Performance Analytics Dashboard

An Interactive Python Data Visualization Project

Internship Project Report

Submitted by

Anurag

B.Tech - Computer Science Engineering (AI & ML)
5th Semester, Third Year

NIELIT Ropar, Punjab, India
Academic Year 2025 - 2026

Table of Contents

(Right-click and select "Update Field" in Word to populate the table of contents and page numbers.)

Certificate

This is to certify that the project report entitled “Student Performance Analytics Dashboard” has been completed by Anurag, a student of B.Tech Computer Science Engineering (AI & ML), 5th Semester, NIELIT Ropar, as part of his internship project work in Python-based data analysis and visualization.

The work presented in this report is based on the design and development of an interactive analytics dashboard built using Python, Pandas, Plotly, and Dash. It reflects a genuine attempt at applying classroom concepts of data handling and visualization to a practical, presentation-ready application, and is found to be satisfactory for the purposes of internship evaluation.

Internship Mentor / Supervisor

Faculty Coordinator

Date: _____

Place: NIELIT Ropar

Declaration

I, Anurag, a student of B.Tech Computer Science Engineering (AI & ML), 5th Semester, hereby declare that the project report titled “Student Performance Analytics Dashboard” is the outcome of my own work carried out during my internship period.

The data structures, code, visualizations, and dashboard layout presented in this report have been designed and implemented by me, with guidance from my internship mentor and faculty wherever required. Any external resources, libraries, or reference material that contributed to this work have been acknowledged in the References section of this report.

This report, in part or in full, has not been submitted earlier for the award of any degree, diploma, or certificate to this or any other institute.

Anurag

B.Tech CSE (AI & ML), 5th Semester

Date: _____

3. Acknowledgement

Working on this dashboard turned out to be one of the more satisfying parts of my internship, mainly because it gave me a chance to combine the data analysis concepts I had studied with an actual visual product that someone could open and interact with.

I would like to thank my internship mentor for the regular feedback and for pointing out, more than once, that a chart which looks fine to the person who built it may not be immediately readable to someone seeing it for the first time. I am also grateful to the faculty at NIELIT Ropar for creating an environment where this kind of applied, project-based learning is encouraged alongside the regular coursework.

My thanks also go to my batchmates, Anand and Ujjawal, for taking the time to click through an early version of the dashboard and pointing out a few layout issues that I had completely missed while testing it only on my own laptop screen.

Finally, I want to acknowledge the documentation and example galleries maintained by the Pandas, Plotly, and Dash open-source communities, which I referred to repeatedly while figuring out how to get the callbacks and the dark theme to behave the way I wanted.

4. Abstract

This report documents the design and development of a Student Performance Analytics Dashboard built entirely in Python. The dashboard brings together two related datasets - one containing academic records such as attendance and marks, and the other covering activity-based

information such as study hours, sports participation, club membership, and assignment completion - into a single interactive interface.

Using Pandas for data handling, Plotly for chart generation, and Dash for the web application layer, the project produces a dark-themed dashboard with summary KPI cards, gender and department filters, and four linked visualizations: a department-wise marks bar chart, a department distribution pie chart, an attendance-versus-marks scatter plot, and a study-hours-versus-marks scatter plot.

The dashboard responds to filter changes in real time, allowing a reviewer to compare departments, genders, and study patterns without writing any additional code. Beyond the technical build itself, this report also covers the data analysis carried out using the dashboard, the difficulties faced while merging and visualizing the two datasets, and the broader lessons learned while building a tool that is meant to be used by someone other than the person who built it.

5. Introduction

Educational institutions generate a fair amount of data every semester - attendance registers, internal assessment marks, final examination scores, assignment submission logs, and records of student participation in sports or clubs. In most cases, this information sits in separate spreadsheets and is rarely examined together. A faculty member might know how a student performed in the final exam, but rarely has an easy way to check whether that performance is connected to attendance, study habits, or co-curricular involvement.

During my internship, I was given the opportunity to work on exactly this kind of problem - building a tool that takes raw student records and turns them into something that can actually be explored rather than just read. The result is the Student Performance Analytics Dashboard, an interactive web application built using Python.

Instead of producing a fixed set of static charts that need to be regenerated every time someone asks a slightly different question, the dashboard lets the user filter by gender and department and instantly see updated marks distributions, department-wise comparisons, and scatter plots relating attendance and study hours to final marks. The dashboard home view, shown later in this report as Figure 1, gives a quick sense of how all of these pieces sit together on one screen.

The project was approached the way a small analytics task inside a company might be approached - start with raw CSV data, clean and merge it, decide which questions are worth answering, build visualizations around those questions, and then wrap everything in an interface that a non-technical reviewer, such as a department head, could use without needing to know any Python.

6. Problem Statement

Most academic performance records are maintained in isolated spreadsheets, and any analysis usually involves manually filtering rows, building pivot tables, and pasting charts into a separate

report. This approach works reasonably well for a one-time summary, but it becomes tedious the moment someone wants to ask a follow-up question - for instance, how the picture changes if only female students in the ECE department are considered. Each new question means repeating the entire manual process from scratch.

There is also the issue of the data itself being split across multiple files. Attendance and marks live in one dataset, while study habits, sports involvement, and assignment completion live in another. Without merging these two sources on a common identifier, it is difficult to see whether students who put in more study hours also tend to have better attendance, or whether attendance and final marks move together at all.

The problem this project tries to address is therefore twofold: first, bring the two datasets together in a meaningful way using a shared Student_ID, and second, present the combined information through an interface that supports filtering and exploration rather than offering only a fixed set of pre-made charts.

7. Project Objectives

The project was planned around the following objectives:

1. To merge and clean two separate student datasets - one academic and one activity-based - into a single working dataset using Pandas.
2. To design summary KPI cards that give an at-a-glance view of total students, average marks, average attendance, and assignment completion rate.
3. To build department-wise visualizations, including a bar chart of student marks and a pie chart showing the overall department distribution.
4. To examine the relationship between attendance and final marks, and between study hours and final marks, through interactive scatter plots.
5. To implement gender and department filters so that every chart and KPI card updates dynamically without reloading the page.
6. To apply a consistent dark theme across the dashboard so that it looks finished and presentation-ready rather than like a quick notebook export.

8. Literature Overview

Before settling on the Python and Dash combination, I spent some time looking at how dashboards are usually built in practice. Tools such as Microsoft Power BI and Tableau are widely used in industry and offer drag-and-drop interfaces that can produce polished dashboards fairly quickly without writing code. Excel, with its pivot tables and conditional formatting, also remains common in smaller institutions simply because everyone already has it installed and knows the basics.

The drawback with Power BI and Tableau, from a student's point of view, is that they are largely closed environments - the logic behind a visualization is hidden inside the tool's interface, and full functionality often sits behind a license. Excel-based dashboards, on the other hand, tend to become harder to maintain once the number of charts, filters, and linked sheets grows beyond a handful.

Python-based dashboards built with Dash, which sits on top of Flask and Plotly, offered a middle ground that suited this project. Every part of the pipeline - from data cleaning to chart styling to the callback logic connecting filters to charts - is written in plain Python, which keeps the entire process transparent and easy to modify. Plotly's charts are also interactive by default, with hover tooltips, zoom, and pan available without any extra effort, which made this combination a natural fit for a project where the goal was exploration rather than a single static report.

9. Tools and Technologies Used

The project relies on a fairly small set of tools, each handling a distinct part of the pipeline, as summarised below.

Technology	Role in the Project
Python 3	Core programming language used for data processing, dashboard logic, and callback functions.
Pandas	Reading the CSV files, cleaning and merging the two datasets, and computing the aggregate statistics shown on the KPI cards.
Plotly Express / Graph Objects	Generating the bar chart, pie chart, and scatter plots, with a dark theme and consistent department colour mapping applied throughout.
Dash	Building the web application layout, wiring up the dropdown filters, and managing real-time updates through callbacks.
Dash HTML & Core Components	Defining the dropdowns, graph containers, KPI cards, and the overall page structure of the dashboard.
Custom CSS (inline styles)	Styling the page background, card layout, fonts, and colours to match the dark theme used in the Plotly charts.

10. System Requirements

Hardware Requirements

Component	Specification
Processor	Intel Core i3 or equivalent (i5 recommended for smoother chart rendering)
RAM	4 GB minimum, 8 GB recommended

Component	Specification
Storage	About 500 MB free space for the project files, datasets, and Python environment

Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux, or macOS
Programming Language	Python 3.9 or later
Libraries	pandas, plotly, dash (installed via pip)
Browser	Google Chrome, Microsoft Edge, or Firefox (the Dash app runs on a local server and is viewed through a browser)
Development Environment	Visual Studio Code, PyCharm, or Jupyter Notebook

11. Dataset Description

Two datasets form the backbone of this project, both containing 100 student records identified through a common `Student_ID` column, which made merging them on a shared key straightforward.

Dataset 1 - Student Performance Data

Field	Description
Student_ID	Unique identifier for each student, used as the key for merging with Dataset 2.
Gender	Recorded as Male or Female.
Department	One of CSE, AIML, or ECE.
Attendance	Percentage attendance for the semester.
Internal_Marks	Marks scored in internal assessments, out of 100.
Final_Marks	Marks scored in the final examination, out of 100.

Dataset 2 - Student Activity Data

Field	Description
Student_ID	Same identifier as in Dataset 1, used to join the two tables.
Sports	Indicates whether the student participates in any sport (Yes/No).
Club	Name of the club the student is associated with, if any.

Field	Description
Study_Hours	Average number of hours the student studies per day.
Assignment_Completion	Percentage of assignments completed on time.

Once merged on Student_ID using an inner join, the combined dataset has 100 rows covering three departments - CSE, AIML, and ECE - in roughly equal proportion, with each department contributing close to a third of the students. This near-equal split is confirmed later by the department distribution pie chart, which shows a 34% / 33% / 33% division across CSE, AIML, and ECE respectively.

12. Methodology

The development process followed a broadly linear path, though there was a fair amount of going back and forth between steps once the first working version of the dashboard was in place.

The first step was loading both CSV files using `pandas.read_csv()` and checking for missing values, duplicate Student_ID entries, and inconsistent column naming. A few rows in Dataset 2 had blank entries in the Club column for students who were not part of any club, so these were filled with the label "None" rather than left empty, since blank cells tend to cause problems during grouping and aggregation later on.

With both datasets cleaned, they were merged using `pandas.merge()` on the Student_ID column with an inner join, producing a single dataframe that contains every field needed for both the academic and activity-based visualizations.

The next step was computing the values shown on the KPI cards - the total number of students using `len()`, the average marks and average attendance using `.mean()` on the respective columns, the overall assignment completion rate, and the department with the highest average marks and highest average attendance, found by grouping on the Department column and comparing the resulting averages.

For the visualizations, Plotly Express was used to build the bar chart (`px.bar`) for department-wise marks, the pie chart (`px.pie`) for department distribution, and the two scatter plots (`px.scatter`) for attendance versus marks and study hours versus marks. Each figure was given a dark template along with a consistent colour scale, so that the same department colours - blue for CSE, orange-red for AIML, and green for ECE - appear across every chart in the dashboard.

Once the static figures were working correctly inside a notebook, the layout was rebuilt inside a Dash application using html and dcc components - Dropdown components for the gender and department filters, and Graph components for each chart. The final and most involved part was writing the callback function, which takes the selected gender and department as inputs and returns updated figures and KPI values as outputs. Testing this callback against different filter combinations, particularly edge cases such as a department with very few female students, took up a noticeable share of the overall development time.

13. Dashboard Architecture

The dashboard is organised as a single-page Dash application with four broad regions stacked vertically, as shown in Figure 1 below.

At the very top sits a row of KPI cards - Total Students, Average Marks, Attendance, Assignments, Top Department, and Best Attendance - each rendered as a separate `html.Div` with its own dark card styling. These cards are recalculated inside the main callback every time a filter changes, so the numbers shown always correspond to the currently selected gender and department.

Below the KPI row sit the two filter dropdowns - Select Gender and Select Department - built using `dcc.Dropdown`. These dropdowns are the only inputs to the main callback; everything else on the page is treated as an output that gets regenerated in response to a change in either dropdown.

The middle section of the page holds the department-wise bar chart on the left and the department distribution pie chart on the right, placed side by side using a flex-based layout. The bottom section contains the two scatter plots - Attendance vs Marks and Study Hours vs Final Marks - split into separate Male and Female views and coloured by department.

From an architecture point of view, the most important decision was keeping all four charts and the KPI values as outputs of a single callback rather than writing a separate callback for each component. This avoided a situation, encountered during early testing, where the bar chart would update on a filter change but the pie chart would still reflect the previous selection because its callback had not fired yet.

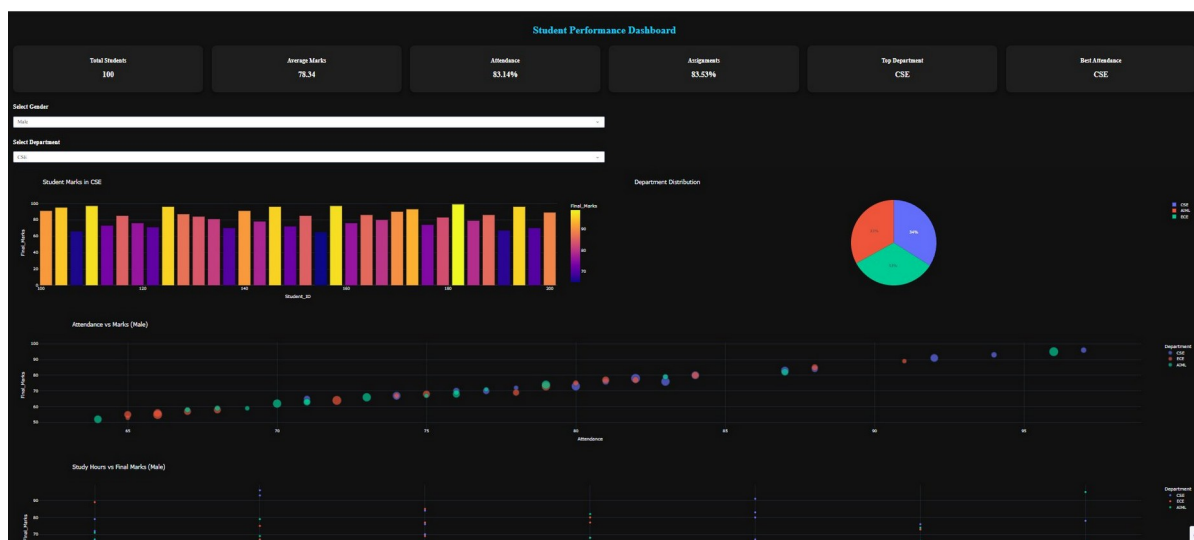


Figure 1: Dashboard home view with KPI cards, filters, and the CSE department charts

14. Implementation Details

At the code level, the project is organised into three logical blocks within a single app.py file - data preparation, figure-generation helper functions, and the Dash layout together with its callback.

The data preparation block reads and merges the two CSV files once, when the application starts, rather than on every callback execution. This was a deliberate choice - recomputing the merge on every filter change would add unnecessary overhead, since the underlying data does not change during a session.

The figure-generation block contains small functions such as `get_bar_chart(df)`, `get_pie_chart(df)`, `get_scatter_attendance(df)`, and `get_scatter_study(df)`. Each of these accepts a dataframe, which may already be filtered, and returns a Plotly figure with the dark theme and department colour mapping already applied. Keeping these as separate functions made the main callback much shorter, since its job becomes simply: filter the dataframe based on the dropdown values, and then pass that filtered dataframe to each of these functions.

A simplified version of the main callback looks roughly like this:

```
@app.callback(
    [Output('bar-chart', 'figure'),
     Output('pie-chart', 'figure'),
     Output('scatter-attendance', 'figure'),
     Output('scatter-study', 'figure'),
     Output('kpi-total', 'children'),
     Output('kpi-avg-marks', 'children')],
    [Input('gender-dropdown', 'value'),
     Input('department-dropdown', 'value')]
)
def update_dashboard(selected_gender, selected_department):
    filtered_df = df.copy()
    if selected_gender:
        filtered_df = filtered_df[filtered_df['Gender'] == selected_gender]
    if selected_department:
        filtered_df = filtered_df[filtered_df['Department'] == selected_department]

    return (get_bar_chart(filtered_df), get_pie_chart(filtered_df),
            get_scatter_attendance(filtered_df), get_scatter_study(filtered_df),
            len(filtered_df), round(filtered_df['Final_Marks'].mean(), 2))
```

The dark theme itself was applied at two levels - Plotly figures use a dark template along with custom paper and plot background colours set to a near-black shade, while the surrounding Dash layout uses inline CSS styles on each html.Div to match the same background. Getting these two shades of “black” to align exactly took a couple of attempts, since Plotly's default dark template background is slightly different from a pure black.

15. Dashboard Features

The finished dashboard combines six KPI cards, two dropdown filters, and four charts into a single connected view.

The KPI cards across the top - Total Students, Average Marks, Attendance, Assignments, Top Department, and Best Attendance - give a quick summary even before the user touches a filter. In the unfiltered view, these show 100 total students, an average final mark of 78.34, an overall attendance figure of 83.14 percent, and an assignment completion rate of 83.53 percent, with CSE coming out on top for both average marks and attendance.

The Select Gender filter restricts the view to Male or Female students, while the Select Department filter narrows the data down to CSE, AIML, or ECE. Figure 2 shows the dashboard after AIML has been selected from the department dropdown - the bar chart immediately relabels itself “Student Marks in AIML” and shows only the AIML students, while the pie chart continues to show the overall department split for context.

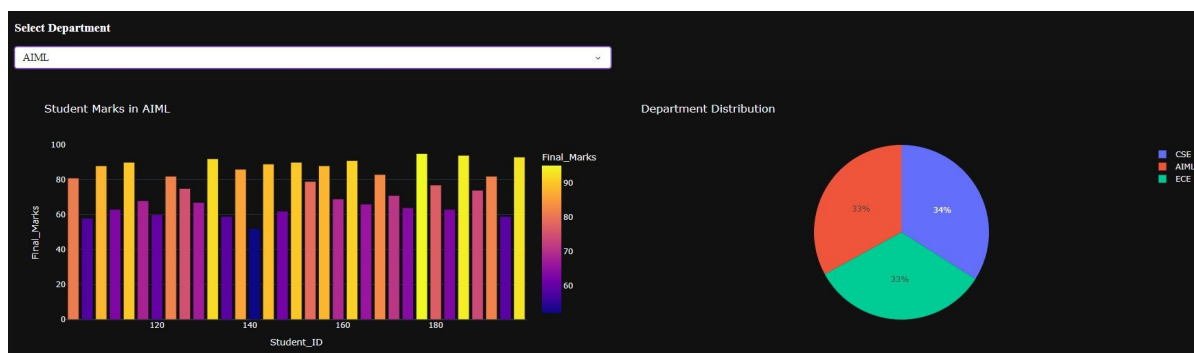


Figure 2: Dashboard view after applying the AIML department filter

The department-wise bar chart uses a continuous colour scale tied to Final_Marks, so within a single department it becomes easy to spot which students scored particularly high or low without reading every bar individually. The pie chart confirms that the three departments are represented in roughly equal proportions in the dataset - useful context when comparing department averages, since none of the departments is being judged on a noticeably smaller sample than the others.

The two scatter plots at the bottom of the dashboard are, in my view, the most analytically useful part of the project, since they move beyond simple counts and averages into actual relationships between variables - a point discussed in more detail in the next section.

16. Data Analysis and Findings

The two scatter plots - Attendance vs Final Marks and Study Hours vs Final Marks - were the part of the dashboard I spent the most time looking at once it was working, mainly because the patterns they reveal are not things one would notice from the KPI cards alone.

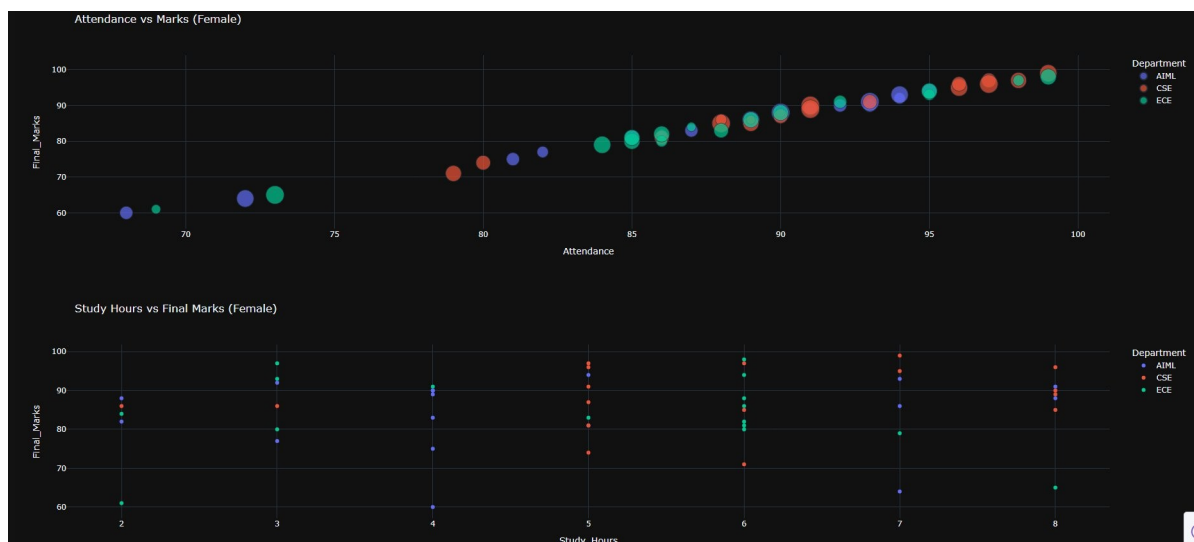


Figure 3: Attendance vs Marks and Study Hours vs Final Marks for female students

Figure 3 shows both scatter plots filtered to female students. In the Attendance vs Marks plot, there is a clear upward trend - students with attendance below 70 percent are clustered around the 60 to 65 mark range, while students with attendance above 90 percent are almost all scoring above 85, with several reaching the high 90s. The relationship is not perfectly linear - a couple of students with attendance around 81 to 82 percent score above 90 - but the overall direction is unmistakable: more consistent attendance lines up with better final marks.

The Study Hours vs Final Marks plot tells a slightly less tidy story. There is still a general upward drift - students studying six to eight hours a day tend to cluster in the 85 to 97 range - but the spread at each study-hour value is noticeably wider than in the attendance plot. At five study hours, for instance, marks range from the mid-70s to the high 90s, and at seven hours one student appears down near 60. This suggests that while study hours matter, they are not the only factor at play; attendance, the quality of those study hours, or other variables not captured in this dataset, such as consistency or exam technique, likely play a role too.

Looking at the colour coding across both scatter plots, ECE and CSE students appear slightly more concentrated in the higher-attendance, higher-marks region compared to AIML in this sample, though the difference is not large given how evenly distributed the departments already are. This lines up with CSE being marked as both the “Top Department” and “Best Attendance” department on the KPI cards.

One pattern worth highlighting is that attendance appears to be a stronger and more consistent predictor of final marks than study hours in this dataset - the attendance scatter plot shows noticeably less vertical spread at each x-value compared to the study hours scatter plot. If this dataset were representative of a real class, it would suggest that interventions aimed at improving attendance might have a more reliable payoff than simply encouraging students to log more study hours, since the latter does not translate into marks as predictably.

17. Results and Observations

Across the unfiltered dataset of 100 students, a few headline numbers stand out, summarised in the table below.

Metric	Value (Overall)
Total Students	100
Average Final Marks	78.34
Average Attendance	83.14%
Assignment Completion Rate	83.53%
Top Performing Department	CSE
Best Attendance Department	CSE
Department Distribution	CSE 34%, AIML 33%, ECE 33%

These figures, taken together, point to a fairly evenly performing cohort, with no department lagging dramatically behind the others - the gap between the top and bottom department in both marks and attendance is in the range of a few percentage points rather than tens of points.

When the gender filter is applied, switching between the Male and Female views shows that female students, in this dataset, tend to occupy the upper portion of both scatter plots more consistently - most female students cluster above 80 marks once attendance crosses 85 percent, with very few low outliers. The male view, visible when CSE is selected, shows a slightly wider spread, including a small cluster of students around 50 to 60 marks despite attendance in the 65 to 70 percent range, which pulls the overall average down slightly for that filtered view.

The department filter, when applied to AIML specifically, shows an average close to the overall figure of 78.34, with a similar spread of bars in the marks chart and no single AIML student standing out as a major outlier in either direction - consistent with AIML's near-equal 33 percent share in the department distribution pie chart.

Overall, the dashboard makes it possible to confirm in a matter of seconds what would otherwise require several separate pivot tables - that CSE leads marginally in both marks and attendance, that the three departments are represented almost equally, and that attendance correlates with final marks more tightly than study hours do.

18. Challenges Faced During Development

A few issues came up repeatedly during development, and most of them turned out to be smaller than they initially seemed but still took longer than expected to resolve.

The first was merging the two datasets cleanly. A handful of Student_ID values in Dataset 2 had leading or trailing whitespace that was not visible when looking at the CSV in a spreadsheet

program, but which caused `pandas.merge()` to silently drop those rows from the inner join. This was only caught by comparing the row counts of both source dataframes against the merged dataframe and noticing that the merged version had fewer rows than expected.

The second challenge was getting the dark theme to look consistent across the dashboard. Plotly's dark template covers the chart area, legend, and axis labels, but the surrounding page background, controlled separately by Dash's own CSS, needed to be set to match. Early versions of the dashboard had charts with a near-black background sitting on a slightly different shade of grey from the rest of the page, which looked unintentional rather than designed.

The third issue was that the scatter plots became cluttered when all departments and both genders were shown together, with points overlapping heavily in the 85 to 95 marks range. Slightly reducing marker opacity, and splitting the scatter plots into separate Male and Female sections rather than trying to fit everything into one chart, helped considerably.

Finally, callback performance became noticeable once all four charts and six KPI values were tied to the same callback - every dropdown change re-rendered everything, including charts that had not visually changed. This was not a serious problem at the 100-row scale used here, but it is something that would need attention if the dataset were to grow significantly larger.

19. Skills Learned

This project ended up touching a wider range of skills than I initially expected when I started.

On the data side, I became much more comfortable with Pandas operations that I had previously only used in isolated classroom exercises - merging dataframes on a key column, grouping and aggregating with `groupby()`, and handling missing or inconsistent values before they cause problems further down the pipeline.

On the visualization side, working with Plotly Express showed me how much can be controlled through a figure's layout and template properties - colour scales, hover templates, and axis titles - without needing to touch the underlying chart type at all.

Building the Dash application itself was probably the steepest part of the learning curve. Understanding the relationship between Input and Output components inside a callback, and the fact that Dash re-runs the entire callback function on every input change rather than just the part that appears to have changed, took some getting used to.

Beyond the purely technical side, I also picked up a better sense of how to read a dataset before building anything on top of it - checking distributions, spotting near-equal splits such as the department percentages here, and learning to tell the difference between a relationship that looks strong and one that is mostly noise with a slight trend, which came up directly while interpreting the scatter plots.

20. Future Scope

The current dashboard works well for a dataset of this size, but there are several directions in which it could be extended further.

One natural addition would be a semester-over-semester view, which would require the dataset to include a time dimension and the dashboard to add a date or semester filter alongside the existing gender and department filters. This would let a reviewer track whether attendance or marks for a particular department are improving or declining over time, rather than looking at a single snapshot.

A second direction would be adding a basic predictive component - using the existing attendance, study hours, and assignment completion fields as inputs to a simple regression or classification model that flags students who might be at risk of scoring below a certain threshold. Since the patterns observed in the scatter plots already hint at attendance being a useful predictor, even a simple linear regression could provide a reasonable starting point for this.

On the interface side, the dashboard could be made more useful to non-technical reviewers by adding a download option that exports the current filtered view, including charts and KPI values, as a PDF or image, rather than requiring the reviewer to take screenshots manually.

Finally, deploying the dashboard to a small cloud platform rather than running it only on a local machine would make it accessible to multiple people at once, which would be a natural next step if a department were to use it for ongoing monitoring rather than as a one-time project.

21. Conclusion

Building this dashboard turned out to be a useful exercise in connecting the separate pieces of a data project - cleaning and merging raw data, deciding which summary statistics and charts actually answer useful questions, and wrapping all of it inside an interface that someone other than the person who built it can use.

The combination of Pandas, Plotly, and Dash proved to be a practical choice for a project of this scale. Pandas handled the data preparation without much friction once the Student_ID mismatch issue was resolved, Plotly produced interactive, dark-themed charts with relatively little custom code, and Dash tied the filters and charts together through a single callback.

From an analysis standpoint, the dashboard surfaced a fairly intuitive but useful finding - attendance tracks more closely with final marks than study hours do, at least in this dataset - while also showing that the three departments are represented almost equally and perform within a fairly narrow band of one another.

Overall, the project met the objectives set out at the start, and the process of building it - particularly debugging the callback logic and getting the dark theme consistent across the page - taught me considerably more about how interactive dashboards work under the hood than I expected when I began.

22. References

1. Pandas Development Team, Pandas Documentation, available at pandas.pydata.org/docs.
2. Plotly Technologies Inc., Plotly Python Graphing Library, available at plotly.com/python.
3. Plotly Technologies Inc., Dash User Guide, available at dash.plotly.com.
4. Python Software Foundation, Python 3 Documentation, available at docs.python.org/3.
5. McKinney, W., Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter, O'Reilly Media.
6. Course material and lab exercises from the B.Tech CSE (AI & ML) curriculum, NIELIT Ropar.